Unit 12: Software Maintenance

CONTENTS

Objectives

After studying this unit, you will be able to:

- discuss software maintenance and software supportability
- describe business process reengineering
- restructuring, economics of reengineering

Introduction

The life of your software does not begin when the code is written and ends when it is released. It instead has a continuous lifespan that stops and starts as needed. Launch marks the beginning of its lifecycle and the start of a significant amount of labour. Software is always changing, and it must be carefully monitored and maintained as long as it is in use. This is mainly to accommodate internal organizational changes, but it is also critical since technology is always evolving.

Your software may require maintenance for a variety of reasons, including keeping it up and running, adding new features, reworking the system to accommodate future changes, moving to the cloud, or making other adjustments. Whatever your incentive for software maintenance is, it is critical to your company's success. As a result, software upkeep entails more than just detecting and addressing flaws. It's about keeping your company's heart beating.

Maintenance may also include adding new features and functionality to an existing software system (using cutting-edge technology). The major goal of software maintenance is to make the software system functional according to the needs of the users and to correct software problems. The issues occur as a result of programme failure or hardware incompatibility with the software. Program

patches are used when only a small portion of the software code needs to be maintained. These patches are only used to correct problems in software code that has errors.

12.1 Software Maintenance

The process of modifying and updating software applications after they have been delivered in order to repair faults and improve performance is known as software maintenance. Software evolution and maintenance improves software performance by Minimising errors, removing unnecessary development, and implementing advanced development techniques.

Software maintenance is a broad term that encompasses optimization, mistake repair, the removal of obsolete functionality, and the augmentation of existing ones. Software maintenance entails making enhancements to a current solution and, on occasion, new software creation to meet changing market demands.

Software maintenance is affected by several constraints such as increase in cost and technical problems with hardware and software. This chapter discusses how software maintenance assists the present software system to accommodate changes according to the new requirements of users.

Let's use the example of an automobile to better grasp the notion of maintenance. When a car is 'used,' its components wear out owing to friction in the mechanical parts, improper use, or environmental factors. The problem is solved by changing the car's components when they become completely unserviceable, and by hiring trained mechanics to tackle complex defects throughout the vehicle's lifetime. The car is occasionally serviced at a service station by the owner. This helps to keep the car in better shape in the future. Similarly, in software engineering, software must be 'serviced' to ensure that it can adapt to changing circumstances (such as business and user needs) in which it operates.

There are number of reasons, why modifications are required, some of them are briefly mentioned below:

Market Conditions - Policies, which changes over the time, such as taxation and newly introduced constraints like, how to maintain bookkeeping, may trigger need for modification.

Client Requirements - Over the time, customer may ask for new features or functions in the software.

Host Modifications - If any of the hardware and/or platform (such as operating system) of the target host changes, software changes are needed to keep adaptability.

Organization Changes - If there is any business level change at client end, such as reduction of organization strength, acquiring another company, organization venturing into new business, need to modify in the original software may arise.

Why Software maintenance

- Bug Fixing
- Capability Enhancement
- Removal of Outdated Functions
- Performance Improvement
- Market Conditions
- Interface with other systems
- Host Modifications

Bug fixing:

In maintenance management, bug fixing comes at priority to run the software seamlessly. This process contains search out for errors in code and corrects them. The issues can be occurred in hardware, operating systems or any part of the software.

Capability Enhancement:

This comprises an improvement in features and functions to make solutions compatible with the varying market environment. It enhances software platforms, work patterns, hardware upgrades, compilers and all other aspects that affect system workflow.

Removal of Outdated Functions:

The unwanted functionalities are useless. Moreover, by occupying space in solution, they hurt efficiency of the solution. Using software maintenance procedures, such UI and coding elements are removed and replaced with new development using the latest tools and technologies.

Performance Improvement:

To improve system performance, developers detect issues through testing and resolve them. Data and coding restricting as well as reengineering are the part of software maintenance. It prevents the solution from vulnerabilities.

Host Modifications:

If any of the hardware and/or platform (such as operating system) of the target host changes, software changes are needed to keep adaptability.

Providing continuity of service:

The software maintenance process focuses on fixing errors, recovering from failures such as hardware failures or incompatibility of hardware with the software, and accommodating changes in the operating system and the hardware.

Supporting mandatory upgrades:

Upgrading a software system is made easier with software maintenance. Changes in government regulations or standards may need upgrades. If a web-application system with multimedia capabilities has been established, for example, it may need to be modified in areas where video viewing (through the Internet) is illegal. Upgrading software may also be necessary to stay competitive with other software in the same category.

Improving the software to support user requirements:

Requirements may be requested to enhance the functionality of the software, to improve performance, or to customize data processing functions as desired by the user. Software maintenance provides a framework, using which all the requested changes can be accommodated.

Facilitating future maintenance work:

Software maintenance also facilitates future maintenance work, which may include restructuring of the software code and the database used in the software.

Changing a Software System

As previously stated, the necessity for software maintenance emerges as a result of changes that must be made to the software system. Anomalies are discovered, new user requirements emerge, and the operating environment changes after a software system has been designed and implemented. This means that, after delivery, software systems are always evolving in response to changing demands.

Lehman was the first to develop the concept of software maintenance and system evolution. He conducted various investigations and offered five laws based on his findings. Large systems are never complete and continue to evolve, according to one of the studies' primary findings. It's worth noting that when systems evolve, they grow more complicated, necessitating some steps to lessen the complexity. The five Lehman laws are presented in Table and explained further below.

Continuing change:

Because systems function in a dynamic context, this law states that change is unavoidable. As the system's environment changes, new requirements emerge, and the system must be adjusted. When the modified system is reintroduced into the environment, it necessitates additional environmental changes. It's worth noting that if a system remains static for a long time, it won't be able to meet the users' ever-changing needs. This is due to the fact that the system may become obsolete over time.

Increasing complexity:

According to this law, as a system evolves, its structure deteriorates (often observed in legacy systems). To avoid this issue, preventive maintenance should be utilized, in which the software's structure is enhanced but no new functionality is added. However, to counteract the effects of structural degradation, extra costs must be paid.

Large software evolution:

Because of organizational elements that are established early in the development process, this law states that software evolution for large systems is mostly dependent on management decisions. This is especially true for major corporations, which have their own internal bureaucracies in charge of decision-making. The rate of change of the system in these organizations is determined by the decision-making processes of the organization. This establishes the overall trends of the system maintenance process and restricts the number of system changes that can be made.

Organizational stability:

This law states that changes to resources such as staffing have unnoticeable effects on evolution. For example, productivity may not increase by assigning new staff to a project because of the additional communication overhead. Thus, it can be said that large software development teams become unproductive because the communication overheads dominate the work of the team.

Conservation of familiarity:

This law indicates that the rate at which new functionality can be added has a limit. This means that adding a significant amount of functionality to a system in a single release will almost certainly result in new system flaws. If a significant increment is introduced, a new release will be necessary 'quite rapidly' to address the new system flaws. As a result, companies should not plan for major functional increments in each release without factoring in the need for bug fixes.

12.2 Types of Software Maintenance

- Corrective maintenance
- Adaptive maintenance
- Perfective maintenance
- Preventive maintenance

Corrective Maintenance:

Detecting errors in the existing solution and correcting them to make it works more efficiently. This type of software maintenance aims to eliminate and fix bugs or defects in the software. Corrective software maintenance is often done in the form of small updates frequently.

Corrective software maintenance is the most common and traditional type of maintenance (for software and anything else for that matter). When something goes wrong with a piece of software, such as bugs or errors, corrective software maintenance is required. These can have a large impact on the software's overall functionality, so they must be addressed as soon as feasible.

Many times, software vendors can address issues that require corrective maintenance due to bug reports that users send in. If a company can recognize and take care of faults before users discover them, this is an added advantage that will make your company seem more reputable and reliable (no one likes an error message after all).

Adaptive maintenance:

Modifications in the system to keep it compatible with changing business and technical environment. This type of software maintenance concentrate on software infrastructure. To retain continuity with the software, adaptive maintenance are made in response to new operating systems, hardware, and platforms.

Adaptive software maintenance is concerned with changing technologies as well as your software's policies and procedures. Changes to the operating system, cloud storage, and hardware are just a few examples. When these modifications are made, your programme must adapt in order to match the new requirements and continue to function effectively.

Perfective maintenance:

Fine tuning of all elements, functionalities and abilities to improve system operations and perfectness. The software's accessibility and usability are solved by perfective software maintenance. Perfective maintenance includes altering current software functionality by improving, removing, or inserting new features or functions.

When software is released to the public, new concerns and ideas emerge, just as they do with any other product. Users may recognise the need for more features or requirements in the software in order to make it the greatest tool for their needs. Perfective software maintenance comes into play at this point. Perfective software maintenance strives to fine-tune software by adding new features when needed and deleting components that are no longer relevant or useful in the current software. As the market and user needs evolve, this approach ensures that software remains relevant.

Preventive maintenance:

Preventive software maintenance services help in preventing the system from any upcoming vulnerabilities. Preventive maintenance means improvements to the software, which is done to secure the software for the future. This maintenance is carried out to prevent the product from any potential software alteration.

Preventative software maintenance entails planning ahead of time to ensure that your programme continues to function as intended for as long as possible. This includes making appropriate adjustments, enhancements, and modifications, among other things. Preventative software maintenance can address minor issues that may seem insignificant at the time but could grow into major concerns in the future. These are known as latent flaws, and they must be identified and repaired so that they do not become active faults.

12.3 Software Maintenance Activities

As per IEEE framework sequential maintenance process activities includes…

Identification & Tracing	Analysis
Design	Implementation
System testing	Acceptance Testing
Delivery	Maintenance management

Identification and Tracing: - It involves activities pertaining to identification of requirement of modification or maintenance. It is generated by user or system may itself report via logs or error messages.

Analysis: - The modification is analysed for its impact on the system including safety and security implications. If probable impact is severe, alternative solution is looked for. A set of required modifications is then materialized into requirement specifications. The cost of modification /maintenance is analysed and estimation is concluded.

Design: - New modules, which need to be replaced or modified, are designed against requirement specifications set in the previous stage. Test cases are created for validation and verification.

Implementation: - The new modules are coded with the help of structured design created in the design step. Every programmer is expected to do unit testing in parallel.

System Testing: - Integration testing is done among newly created modules. Integration testing is also carried out between new modules and the system. Finally the system is tested as a whole, following regressive testing procedures

Acceptance Testing: - After testing the system internally, it is tested for acceptance with the help of users. If at this state, user complaints some issues they are addressed or noted to address in next iteration.

Delivery: - After acceptance test, the system is deployed all over the organization either by small update package or fresh installation of the system. The final testing takes place at client end after the software is delivered.

Maintenance management: - Configuration management is an essential part of system maintenance. It is aided with version control tools to control versions, semi-version or patch management.

12.4 Software Supportability

Software supportability refers to a software system's ability to be supported during its entire product lifecycle. Software supportability refers to a set of software design attributes, related development tools and methodologies, and environmental infrastructure that make it possible to accomplish software support tasks.

This includes not only meeting any necessary needs or requirements, but also providing the necessary equipment, support infrastructure, new software, facilities, labour, or other resources to keep the programme functioning and capable of performing its purpose.

Supportability is the ability of a complete system design to support operations and readiness needs at a reasonable cost throughout the system's life cycle. It allows for the evaluation of a complete system design's suitability for a set of operational requirements within the anticipated operations and support environment.

Software Support encompass

Key aspects associated to the software

- Operation
- Logistics Management
- Modification

Operation:

Operation covers all aspects associated to the actual use of the software, including the installation, loading (or unloading), configuration, error recovery and execution of the software.

Logistics Management:

Logistics Management covers all aspects related to the handling of the software once a new baseline has been produced, until its delivery to the end user.

Modification:

Modification covers all aspects related to the evolution of the software due to the need of fixing bugs, or adding/changing functionality due to changing user needs.

Software supportability process

- Program planning and control
- Mission, development & support systems definition
- Preparation and evaluation of alternatives
- Determination of SAS requirements
- Software supportability assessment

Program planning and control:

- Development of an early Software supportability Strategy
- Program and Design Reviews

Mission, development & support systems definition:

- Use Study
- Technological Opportunities
- Standardization
- Design Factors
- Comparative Analysis
- Integration of ADP Systems

Preparation and evaluation of alternatives:

- Functional and Non-Functional Requirements
- Support System Alternatives
- Evaluation of Alternatives & Trade-Offs

Determination of SAS requirements:

- Operational Task Analysis

- Software Exception/Problem Support Analysis
- Software Modification Analysis
- Software Transition Analysis
- Post-Deployment Software Support/Logistics Management Analysis

Software supportability assessment:

- Operation
- Modification
- Problem Reaction
- Logistics Management
- Lessons Learned

Software Supportability Principles

Designing for supportability requires diligence on the part of both managers and engineers

What can managers do to identify vulnerabilities?

"What if" scenarios

Ask your technical team what the Achilles heel is – They will tell you!

- What can engineers to improve supportability through design?
- Use a fully replicated production environment for pre-release testing
- Parameterize using configuration files
- Use frameworks to control design
- Carefully evaluate COTS products before incorporating into the design
- Incorporate distributed component design up front

Check the "ilities":

- Security
- Reliability
- Flexibility
- Maintainability
- Scalability
- Availability

Manage change:

- Don't attempt too much change at once
- Evaluate system impacts with changing requirements
- Use the CCB (Configuration Control Board)
- Resist the temptation to "just add it in this time"

Quality Control:

- Monitor to maintain quality and identify new risks

  Keep CMMI inspections technical

  Develop processes and follow them

- Enforce Independent Verification and Validation

  At a minimum, developers should not test their own code

- QA person should report to Program Manager

Organize for Supportability:

- Supportability failures often occur between teams or areas of expertise
  i.e., software team, network team, SA, Security, etc.
- Mitigation strategy

  Assign someone the specific role of enforcing cross- disciple technical quality

12.5 Factors Affecting Software Supportability

Change traffic: Changing traffic is a complex function of requirement stability, software integrity and system usage. Changing traffic will affect the task quantity of software support, and changing traffic will require more software modification work.

Expansion: Extension ability is an attribute related to system design, and insufficient extension ability may limit the scope of software modification and have an important impact on the cost of modification.

System number and deployment location: The number and distribution of the system will have a great impact on the cost of support, but also affect the formulation of software support requirements and software support workload. It also affects the distribution of software support facilities.

Modularization: Modularization facilitates the implementation of software support, and the poor degree of modularization usually leads to an increase in the cost of modification, as other parts of the software need to be modified accordingly.

Software scale: The size of the software has an impact on change traffic, in addition to the amount of resources required to make changes.

Confidentiality: The secret level of data, executable code and documents may impose restrictions and requirements on software support activities, as well as special processing requirements on the system. These effects will limit the use of software and put forward corresponding requirements for design, thus requiring special software support tasks and equipment.

Personnel skills: The software modification needs the personnel who have the corresponding software engineering skill to complete, the software personnel skill level cannot reach the request will affect the software support realization, at the same time, the stipulation skill also will have the influence to the training request.

Standardization: Software development processes and related software documentation to meet standardization requirements can reduce the variety of tools, skills and facilities required and help improve software supportability.

Documentation: Documentation refers to all records, including electronic and paper records, i.e. requirements, design, implementation, testing and using documents related to software products. Documentation is the main factor affecting software supportability.

12.6 Reengineering

It is process of redesign of business processes and the associated systems and organizational structures to achieve a dramatic improvement in business performance.

When we need to update the software to keep it to the current market, without impacting its functionality, it is called software re-engineering. It is a thorough process where the design of software is changed and programs are re-written. Legacy software cannot keep tuning with the latest technology available in the market. As the hardware become obsolete, updating of software becomes a headache. Even if software grows old with time, its functionality does not.

Motivations for reengineering

- Reduce costs/expenses
- Improve financial performance
- Reduce external competition pressure
- Reverse erosion of market share
- Respond to emerging market opportunities
- Improve customer satisfaction
- Enhance quality of products and services

Reengineering project guidelines

- Planning and launching
- Current state assessing and learning from others
- Designing solution
- Developing business case justification
- Developing solution
- Implementing solution

12.7 Business Process Reengineering

Business process re-engineering is the radical redesign of business processes to achieve dramatic improvements in critical aspects like quality, output, cost, service, and speed. Business process reengineering (BPR) aims at cutting down enterprise costs and process redundancies on a very huge scale.

Business process reengineering is the act of recreating a core business process with the goal of improving product output, quality, or reducing costs. It involves the analysis of company workflows, finding processes that are sub-par or inefficient, and figuring out ways to get rid of them or change them.

Steps of business process reengineering

- Map the current state of your business processes
- Analyze them and find any process gaps or disconnects
- Look for improvement opportunities and validate them
- Design a cutting-edge future-state process map
- Implement future state changes and be careful of dependencies

Map the current state of your business processes:

Gather data from all resources–both software tools and stakeholders. Understand how the process is performing currently.

Analyze them and find any process gaps or disconnects:

Identify all the errors and delays that hold up a free flow of the process. Make sure if all details are available in the respective steps for the stakeholders to make quick decisions.

Look for improvement opportunities and validate them:

Check if all the steps are absolutely necessary. If a step is there to solely inform the person, remove the step, and add an automated email trigger.

Design a cutting-edge future-state process map:

Create a new process that solves all the problems you have identified. Don't be afraid to design a totally new process that is sure to work well. Designate KPIs for every step of the process.

Implement future state changes and be careful of dependencies:

Inform every stakeholder of the new process. Only proceed after everyone is on board and educated about how the new process works. Constantly monitor the KPIs (Key Performance Indicators).

12.8 Software Reengineering

Software Reengineering is the process of updating software without affecting its functionality. It is a process of software development which is done to improve the maintainability of a software system. Re-engineering is the examination and alteration of a system to reconstitute it in a new form.

This process may be done by developing additional features on the software and adding functionalities that may or may not be required but considered to make the software experience better and more efficient. It affects positively at software cost, quality, service to the customer and speed of delivery.

Need of software Re-engineering

Processes in continuity: The functionality of older software product can be still used while the testing or development of software.

Boost up productivity: Software reengineering increase productivity by optimizing the code and database so that processing gets faster.

Reduction in risks: Instead of developing the software product from scratch or from the beginning stage here developers develop the product from its existing stage to enhance some specific features that are brought in concern by stakeholders or its users.

Drastic change in technology: The situation when the initially promising technology is replaced by more successful and advanced alternatives is common in IT. The market is constantly evolving and, if the company wants to keep up with technology, reengineering process becomes a necessity.

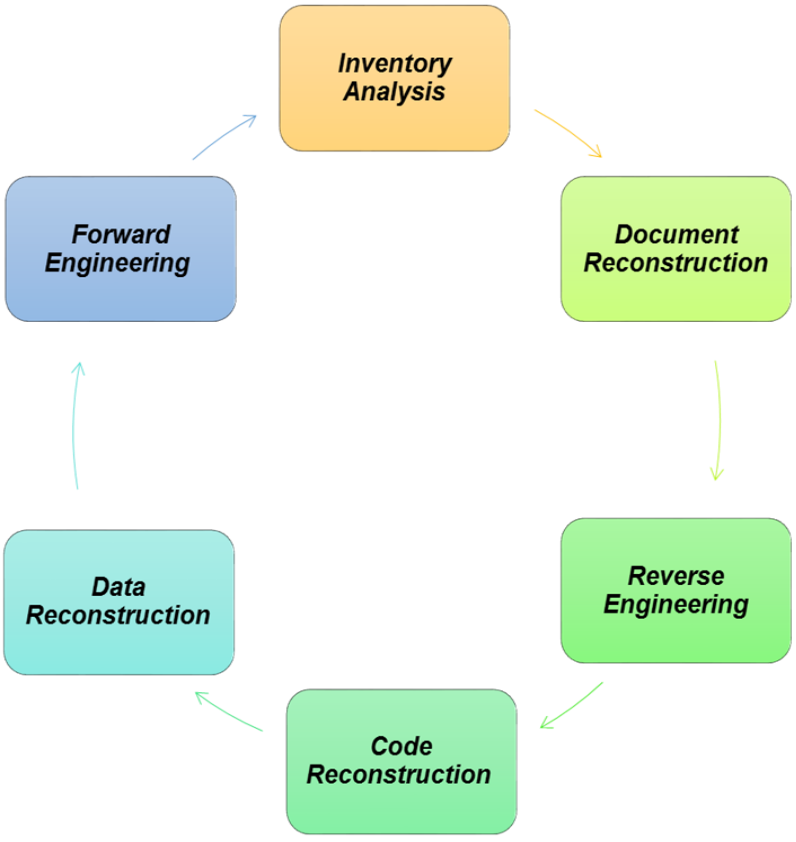
Saves time: As we stated above here that the product is developed from the existing stage rather than the beginning stage so the time consumes in software engineering is lesser.

Optimization: This process refines the system features, functionalities and reduces the complexity of the product by consistent optimization as maximum as possible.

This is an approach by which you can find useless steps and resources that have been implemented in the latest software and then you can remove them from the application

12.9 Steps for Software Re-Engineering

- Inventory Analysis
- Document Reconstruction
- Reverse Engineering
- Code Reconstruction
- Data Reconstruction
- Forward Engineering



Inventory Analysis:

Inventory containing information that provides a detailed description of every active application. By sorting this information according to business criticality, longevity, current maintainability and other local important criteria, candidates for re-engineering appear. The resource can then be allocated to a candidate application for re-engineering work.

Document reconstructing:

- Documentation of a system either explains how it operates or how to use it.
- Documentation must be updated.
- It may not be necessary to fully document an application.
- The system is business-critical and must be fully re-documented.

Reverse Engineering:

Reverse engineering involves the exploration of the existing system to recapture current requirements (including all connections and interdependencies), interfaces between software components, data structure, and data design.

To extract the lost info about the application state, business analysts interview stakeholders, and programmers, study the existing application documentation, perform its lexical and syntactic code analysis, investigate control and data flows, explore use and test cases.

Code Reconstructing:

To accomplish code reconstructing, the source code is analysed using a reconstructing tool. Violations of structured programming construct are noted and code is then reconstructed. The resultant restructured code is reviewed and tested to ensure that no anomalies have been introduced.

Data Restructuring:

Data restructuring begins with a reverse engineering activity. Current data architecture is dissected, and the necessary data models are defined. Data objects and attributes are identified, and existing data structure are reviewed for quality.

Forward Engineering:

Forward engineering also called as renovation or reclamation not only for recovers design information from existing software but uses this information to alter or reconstitute the existing system in an effort to improve its overall quality.

Advantages of Re-engineering

Productivity increase. By optimizing the code and database the speed of work is increased.

Improvement opportunity. You can not only refine the existing product, but also expand its capabilities by adding new features.

Risk reduction. Development from scratch is always a more risky exercise, as opposed to a phased upgrade of the existing system.

Time saving. Instead of starting development from scratch, the existing solution is simply transferred to a new platform, saving all business-logic.

Optimization potential. You can refine the system functionality and increase its flexibility, ensuring better compliance with the enterprise's current objectives.

Processes continuity. The old product can be used while testing the new system until all work is completed.

12.10 Economics of Reengineering and Restructuring

Economics of Re-Engineering

The objective of re-engineering is to reduce maintenance costs. It can achieved by increasing quality and reducing complexity. The costs of re-engineering are driven by the size and interdependency of the software as well as by the degree of automation available.

Parameters for cost/benefit analysis model for reengineering

- P1 = current annual maintenance cost for an application.
- P2 = current annual operation cost for an application.
- P3 = current annual business value of an application.
- P4 = predicted annual maintenance cost after reengineering.
- P5 = predicted annual operations cost after reengineering.
- P6 = predicted annual business value after reengineering.

- P7 = estimated reengineering costs.
- P8 = estimated reengineering calendar time.
- P9 = reengineering risk factor (P9 = 1.0 is nominal).
- L = expected life of the system.

The cost associated with continuing maintenance of a candidate application

$$C_{maint} = [P3 - (P1 + P2)] \times L$$

The costs associated with reengineering are defined using the following relationship

$$C_{reeng} = [P6 - (P4 + P5) \times (L - P8) - (P7 \times P9)]$$

Using the costs presented in equations, the overall benefit of reengineering can be computed as

$$\text{Cost benefit} = C_{reeng} - C_{maint}$$

Cost of maintenance = cost annual of operation and maintenance over application lifetime

Cost of reengineering = predicted return on investment reduced by cost of implementing changes and engineering risk factors

Cost benefit = Cost of reengineering - Cost of maintenance

The cost/benefit analysis presented in the equations can be performed for all high priority applications identified during inventory analysis. Those application that show the highest cost/benefit can be targeted for reengineering, while work on others can be postponed until resources are available.

Software Restructuring

Software restructuring is a form of perfective maintenance that modifies the structure of a program's source code. Its goal is increased maintainability to better facilitate other maintenance activities, such as adding new functionality to, or correcting previously undetected errors within a software system.

Restructuring of software is usually performed in either higher or lower levels of granularity, where the first indicates broader changes in the system's structural architecture and the latter indicates re-factorings performed to fewer and localized code elements.

Types of Restructuring

Code restructuring

Program logic modeled using Boolean algebra and series of transformation rules are applied to yield restructured logic. Create resource exchange diagram showing data types, procedure and variables shared between modules, restructure program architecture to minimize module coupling.

Types of Restructuring

- Data restructuring
- Analysis of source code
- Data redesign
- Data record standardization
- Data name rationalization
- File or database translation

Summery

Maintaining a system is not less important than Web Application Development. It keeps solutions athletic to deal with developing technology and business environment.

Re-Engineering: Restructuring or rewriting part or all of a system without changing its functionality.

Applicable when some (but not all) subsystems of a larger system require frequent maintenance. Reengineering involves putting in the effort to make it easier to maintain. The reengineered system may also be restructured and should be re-documented.

Types of maintenance

Corrective Software Maintenance

Adaptive Software Maintenance

Perfective Software Maintenance

Preventive Software Maintenance

Keywords

Software Maintenance
Restructuring
Forward Engineering
Data Reconstruction
Software reengineering
Economics of reengineering
Code Reconstruction

Self Assessment

1. Software maintenance is used for__
A. Removal of Outdated Functions
B. Performance Improvement
C. Market Conditions
D. Above all

2. Which is not type of Software Maintenance?
A. Unit maintenance
B. Adaptive maintenance
C. Perfective maintenance
D. Above all

3. Software Maintenance Activities includes_
A. Design
B. Implementation
C. System testing
D. Above all

4. Software Support encompass__
A. Operation
B. Logistics Management
C. Modification
D. Above all

5. Software supportability process not include__
A. Program planning and control
B. System testing
C. Preparation and evaluation of alternatives
D. Determination of SAS requirements

6. What are the Software Supportability Principles
A. Check the "ilities"
B. Manage Change
C. Control Quality
D. Above all

7. What are the Steps of business process reengineering?
A. Analyze them and find any process gaps or disconnects
B. Look for improvement opportunities and validate them
C. Design a cutting-edge future-state process map
D. Above all

8. Which is not part of Reengineering project guidelines__
A. Designing solution
B. Developing business case justification
C. Redesign the system
D. Developing solution

9. What are the Motivations for reengineering?
A. Reduce external competition pressure
B. Reverse erosion of market share
C. Respond to emerging market opportunities
D. Above all

10. What are the parameters for Economics of Re-Engineering?
A. Current annual maintenance cost for an application.
B. Current annual operation cost for an application.
C. Estimated reengineering costs
D. Above all

Answer for Self Assessment

1. D	2. A	3. D	4. D	5. B
6. D	7. D	8. C	9. D	10. D

Review Question

1. What is significance of software maintenance?
2. Why software maintenance is required?
3. Differentiate between Corrective maintenance and Adaptive maintenance.
4. Differentiate between Perfective maintenance and Preventive maintenance.
5. Explain different Software Maintenance Activities
6. Discuss the concept of Software reengineering.
7. Define Forward Engineering.

Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering—A Practitioner's Approach, 5th Edition, Tata
- McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education.

Online Links

- http://www.scribd.com
- http://en.wikipedia.org
- https://radixweb.com/blog/why-software-maintenance-is-necessary
- https://www.tutorialspoint.com/software_engineering/software_maintenance_overview.htm
- https://www.javatpoint.com/software-engineering-software-maintenance
- https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance